

RICORSIONE

↳ Metodo alternativo alla costruzione di procedure iterative

Procedura ricorsiva: una procedura è ricorsiva se è definita in funzione di se stessa
↳ se all'interno della procedura esiste una chiamata alla procedura stessa

```
int fact (int m) {  
    if (m ≤ 1) then return m;  
    else return m * fact (m-1);  
}
```

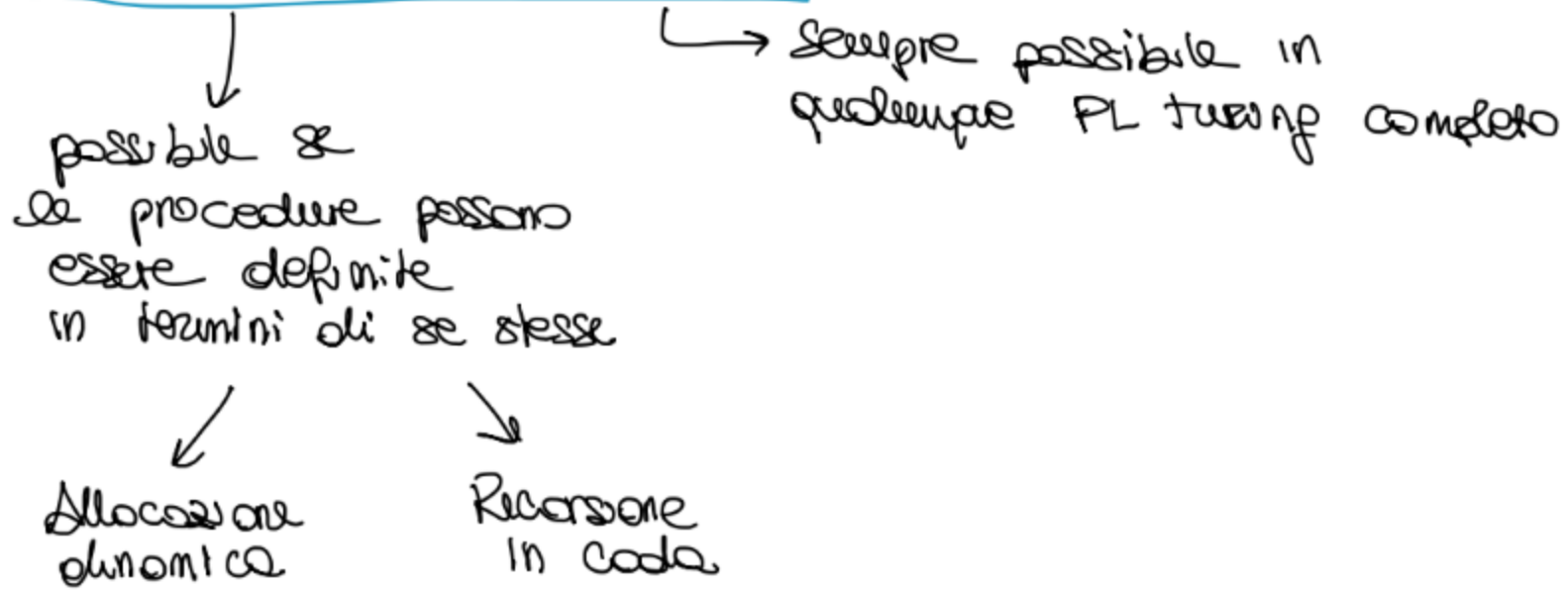
$$\text{fact}(m) = \begin{cases} m & m \leq 1 \\ m * \text{fact}(m-1) & \text{altrimenti} \end{cases}$$

```
int fun1 (int m) {  
    if (m == 1) then return fun1(m)  
}
```

chiamata ricorsiva
→ chiamata alla funzione

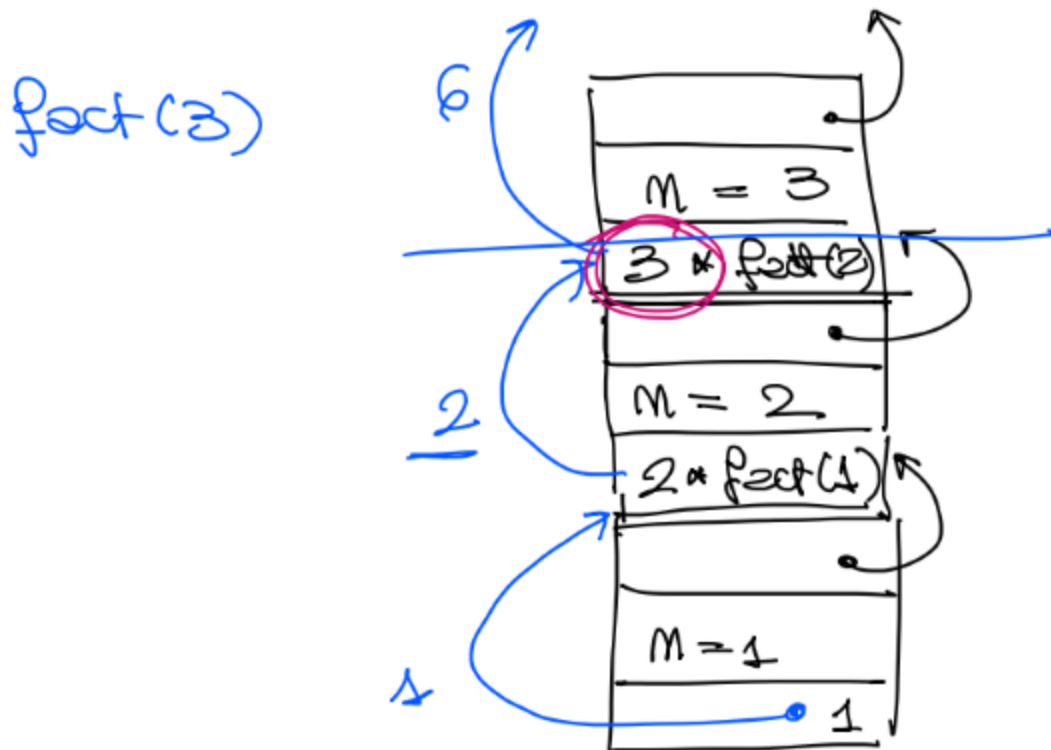
```
int fun2 (int m) {  
    if (m == 0) then return 1;  
    else fun2(m) + 1;  
}
```

Recorsione vs Iterazione



→ Ogni programma iterativo può essere scritto in modo
ricorsivo e viceversa ogni programma iterativo
può essere trasformato in forma ricorsiva

```
int fact (int n) {  
    if (n <= 1) return n;  
    else return n * fact (n-1);  
}
```



Recozzione in coda

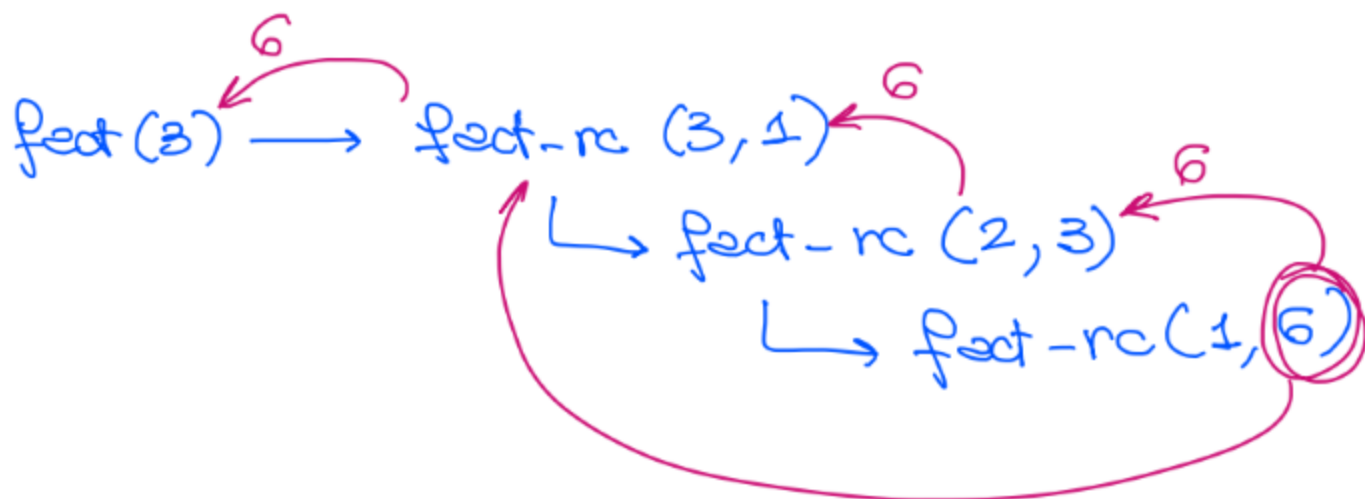
- Chiamata in coda : . f chiama g
- la chiamata di g è detta in coda se f restituisce il valore restituito da g senza ulteriori computazioni
- f è ricorsiva in coda : se la chiamata ricorsiva a se stessa è in coda

parametro $m \geq 1$

```
int fact (int m) {  
    fact-rc (m, 1);  
}
```

valore iniziale per res → indicativamente è il risultato del caso base

```
int fact-rc (int m, int res) {  
    if (m == 1) return res;  
    else return fact-rc (m-1, m * res);  
}
```



Esempio - Fibonacci

$$\begin{cases} \text{Fib}(0) = 1 \\ \text{Fib}(1) = 1 \\ \text{Fib}(n) = \text{Fib}(n-1) + \text{Fib}(n-2) \\ n \geq 2 \end{cases}$$

```
int fib(int n) {  
    if (n <= 1) return 1;  
    else return fib(n-1) + fib(n-2);  
}
```

→ complessità esponenziale
sia in tempo che in
spazio

```
int fib(int n) {  
    fib-rc(n, 1, 1);  
}
```

base per $n=0$

base per $n=1$

Valori inizializzati e valori
che permettono di restituire
subito il risultato del caso base

risultato
del caso
precedente

risultato calcolato
attuale

```
int fib-rc(int n, int res1, int res2) {
```

```
    if (n <= 1) return res2;
```

```
    else return fib-rc(n-1, res2, res1 + res2);
```

```
}
```

Esercizi sulla Escursione

Chiede di trasformare una procedura ricorsiva data in una procedura ricorsiva in code.

↳ def breve di risonanza

↳ def breve di ricorrenza in code

↳ esecuzione della procedura data sull'input specificato

↳ breve descrizione e parole di cose
e la procedura, ovvero come trasforma
l'input

→ Scrittura della procedura ricorsiva in code
con chiamate multiple (→ inizializzazione di res)

↳ esecuzione delle procedure scritte sull'input specificato

Programma wczasy w ostate:

list2 fun (list2 list) of

if length (list) = 0 return list;

else return

concat (fun (cdr (list)), car (list));
concatenazione coda della testa della

concurrent zone

codice della
lista

festa delle
listig

28/11/2020
uma lista

Testifica
un elemento

Esequire su $(3, 6, 9)$

(9, 6, 3)

$$\text{fun}((\underbrace{3, 6, 9}_{\text{list}})) \rightarrow \text{concat}(\underbrace{\text{fun}((6, 9))}_{\text{list}}, 3)$$

28

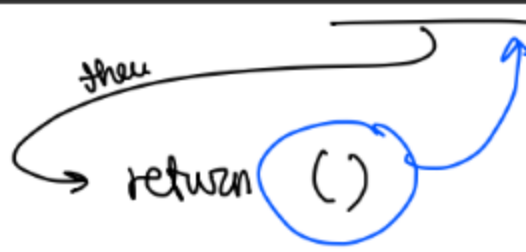
→ concat (fun (q)), 6)

 $(9, 6)$

else

↪ concat (fun (c), q) (q)

 (a)



⇒ Il programma esegue una reverse della lista

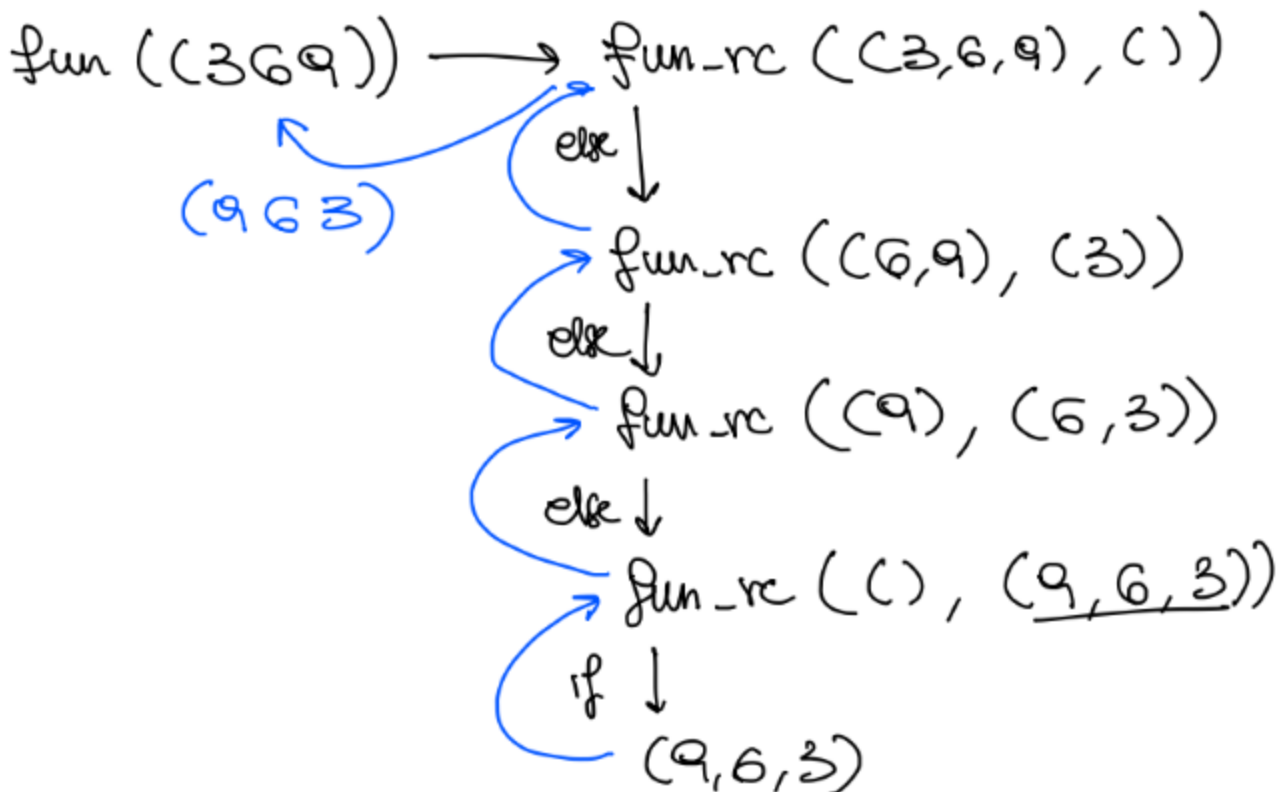
Dobbiamo fornire la versione ricorsiva in code

```
lista fun (lista list) {  
  fun-rc (list, ());  
}
```

lista vuota

```
lista fun-rc (lista list, lista res) {  
  if length(list)=0 return res;  
  ( )  
  else return  
    fun-rc (cdr(list), concat(car(list), res));  
}
```

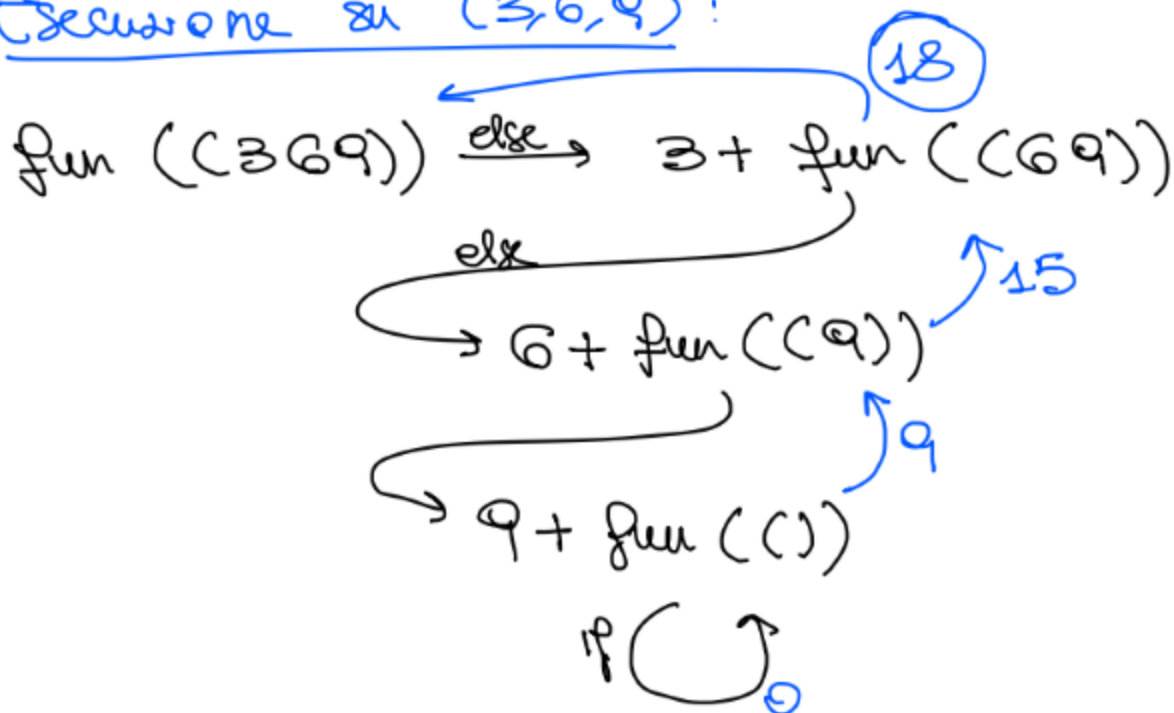
Esecuzione su (3,6,9)



Esempio 2

```
int fun (liste list) {  
    if length(list) == 0 return 0;  
    else return (car(list) + fun(cdr(list)));  
}
```

Esecuzione su (3, 6, 9):



→ la funzione calcola la somma degli elementi che compongono la lista

Conversione in code

```
int fun (liste list) {  
    return fun-rc (list, 0);  
}
```

```
int fun-rc (liste list, int res) {
```

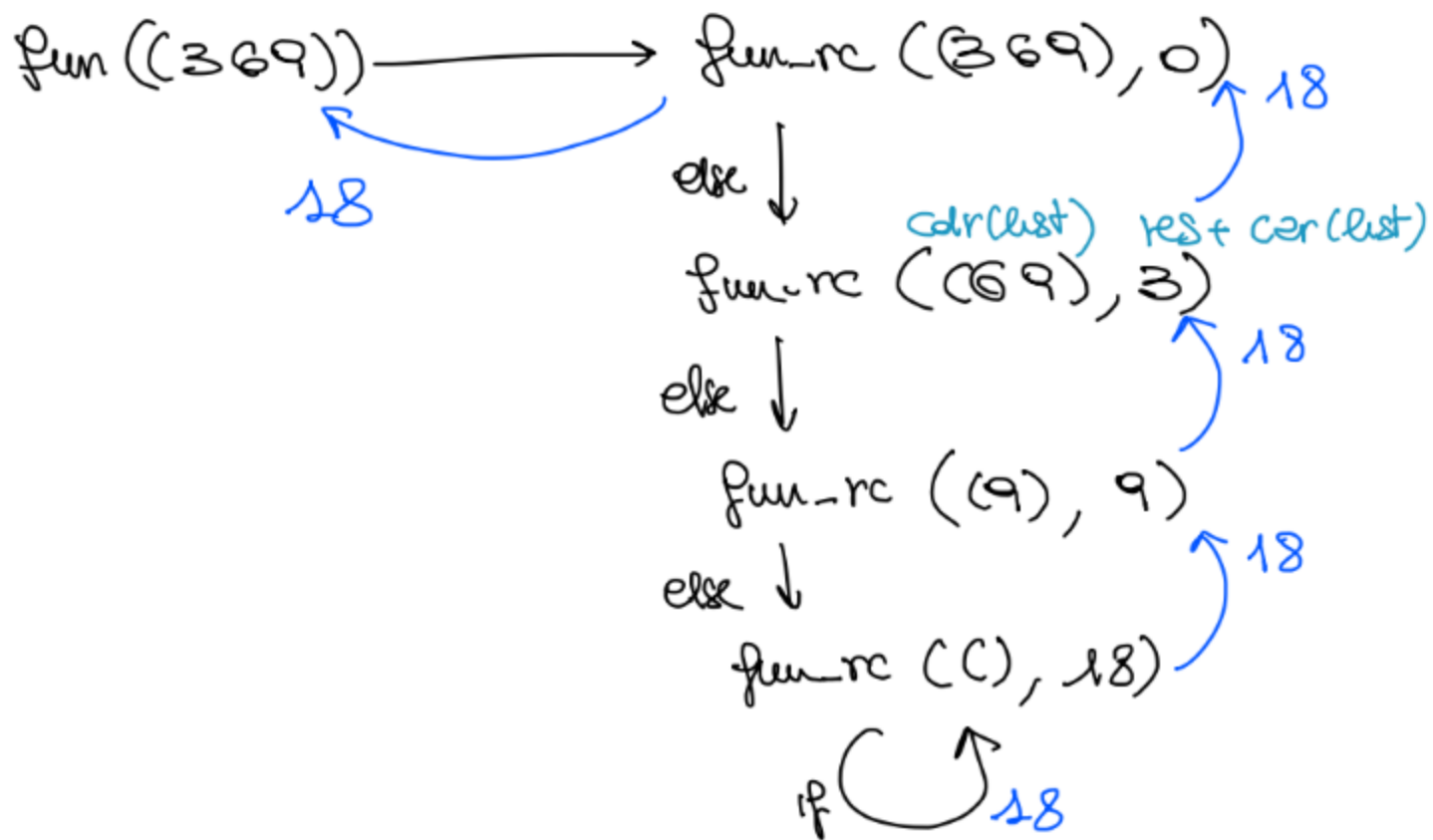
```
    if (length(list) == 0) return res;
```

```
    else return fun-rc (cdr(list), res + car(list));
```

```
}
```

← dove essere del tipo del valore di ritorno

Esecuzione su (369)



Esercizio (da fare)

```
def fun (list: list):  
  if length(list) == 0 return ();  
  else return  
    concat (2 * car(list), fun (cdr(list)));  
  |
```

Esempio su (3,6,9) e rendere ricorsivo in code.